FIBONACCI SERIES IN DATA STRUCTURES

Customer Loyalty Reward System

Group Members

Group 3

1.Brandon Kigen

2.Chriss Wambua

3.Sydney Langat

4.Prince Munene

5.John Victor

Submission Date: 16th October 2026

CASE STUDY REPORT

This report designs and analyzes different approaches for generating the Fibonacci series using data structures, and applies them to a customer loyalty reward system.

1. RECURSIVE APPROACH

1.1 Implementation

```
def fibonacci_recursive(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)

def fibonacci_series_recursive(n):
    return [fibonacci_recursive(i) for i in range(n + 1)]

print(fibonacci_series_recursive(10))
```

1.2 How it works

- The function calls itself to calculate smaller Fibonacci numbers.
- Base cases: F0 = 0 and F1 = 1.
- Recursive case: F(n) = F(n-1) + F(n-2).
- Example: fib(5) calls fib(4) and fib(3).
- The same values, such as fib(3), are calculated more than once.

1.3 Time complexity

- Time complexity: O(2^n), exponential time.
- The function recalculates the same subproblems many times.
- This creates overlapping subproblems.
- As n increases, the number of calculations grows very quickly.

1.4 Space complexity

- Space complexity: O(n).
- The recursion stack may hold up to n function calls.

1.5 Why recursion may become inefficient

- It repeats the same calculations many times.
- Its execution time grows exponentially.
- It becomes slow for large values of n.
- It is not suitable for a loyalty system serving thousands of customers.

2. ITERATIVE APPROACH

2.1 Implementation using arrays

```
def fibonacci_iterative(n):
    if n < 0:
        raise ValueError("n must be non-negative")
```

```
    series = []
    for i in range(n + 1):
        if i == 0:
            series.append(0)
        elif i == 1:
            series.append(1)
        else:
            series.append(series[i - 1] + series[i - 2])
    return series

print(fibonacci_iterative(10))
```

2.2 How it works

- The method uses a loop instead of repeated recursive calls.
- It starts with 0 and 1.
- Each new Fibonacci number is calculated by adding the previous two numbers.
- All results are stored in an array.
- Each Fibonacci number is calculated only once.

2.3 Time and space complexity

- Time complexity: O(n).
- Space complexity: O(n) because the array stores the complete series.
- If only the latest value is needed, two variables can be used, giving O(1) space.

2.4 Comparison with recursion

- Recursive: O(2^n) time.
- Iterative: O(n) time.
- Iterative avoids repeated calculations.
- Iterative is much faster and more scalable.

3. DYNAMIC PROGRAMMING APPROACH

3.1 Memoization

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fibonacci_memoized(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 0
    if n == 1:
        return 1
    return fibonacci_memoized(n - 1) + fibonacci_memoized(n - 2)

def fibonacci_series_memoized(n):
    return [fibonacci_memoized(i) for i in range(n + 1)]
```

- Memoization is a top-down dynamic-programming technique.
- It stores already-calculated results in a cache.
- Before calculating a value, it checks whether the value is already stored.
- Time complexity: O(n).

- Space complexity: O(n).

3.2 Tabulation

```
def fibonacci_tabulation(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return [0]
    if n == 1:
        return [0, 1]
    table = [0] * (n + 1)
    table[1] = 1
    for i in range(2, n + 1):
        table[i] = table[i - 1] + table[i - 2]
    return table
```

- Tabulation is a bottom-up dynamic-programming technique.
- It starts from F0 and F1.
- It fills a table up to Fn.
- Time complexity: O(n).
- Space complexity: O(n).

3.3 Space-time trade-offs

- Dynamic programming uses extra memory to avoid repeated work.
- Memoization and tabulation reduce time from O(2^n) to O(n).
- Both use O(n) extra space.
- This is a good trade-off for a scalable loyalty system.
- If only the latest reward value is needed, an optimized iterative method can use O(1) space.

4. APPLICATION PROBLEM: CUSTOMER LOYALTY REWARD SYSTEM

4.1 Problem model

- Input: number of years a customer has been loyal.
- Output: reward points for each year following the Fibonacci sequence.
- Storage: reward points are stored in an array.
- Display: the program displays each year and its reward points.

4.2 Implementation

```
def loyalty_rewards(years):
    if years < 0:
        raise ValueError("Years of loyalty cannot be negative")
    rewards = []
    for year in range(years + 1):
        if year == 0:
            rewards.append(0)
        elif year == 1:
            rewards.append(1)
        else:
            rewards.append(rewards[year - 1] + rewards[year - 2])
    return rewards
```

```
def display_rewards(years):
    rewards = loyalty_rewards(years)
    print("Customer Loyalty Reward System")
    print(f"{'Year':<10}{'Reward Points':<15}")
    for year, points in enumerate(rewards):
        print(f"{year:<10}{points:<15}")
    print(f"Total reward points after {years} years: {rewards[-1]}")

display_rewards(10)
```

4.3 Sample output for 6 years

- Year 0: 0 reward points
- Year 1: 1 reward point
- Year 2: 1 reward point
- Year 3: 2 reward points
- Year 4: 3 reward points
- Year 5: 5 reward points
- Year 6: 8 reward points

5. PERFORMANCE ANALYSIS FOR n = 30

5.1 Results

- Recursive: 0.09115389 seconds.
- Iterative with array: 0.000001728 seconds.
- Memoization: 0.0000000805 seconds.
- Tabulation: 0.000001489 seconds.

5.2 Interpretation

- Recursive Fibonacci is much slower because it repeats calculations.
- Iterative, memoization, and tabulation methods are extremely fast.
- The exact times may differ on another computer, but the pattern remains the same.

5.3 Recommendation

- Use the iterative approach with an array.
- It has O(n) time complexity.
- It avoids repeated calculations.
- It stores reward points for every loyalty year.
- It has no recursion-stack overhead.
- It is simple, reliable, and scalable.
- Therefore, it is the most suitable approach for the customer loyalty reward system.

CONCLUSION

The recursive method directly follows the Fibonacci definition but becomes inefficient because of repeated calculations. The iterative method using an array calculates each Fibonacci number only once and stores the complete reward history. Dynamic programming further avoids repeated work through memoization or tabulation. For the customer loyalty reward system, the iterative array-based method is recommended

because it is fast, scalable, simple, and able to display reward points for every year of customer loyalty.